

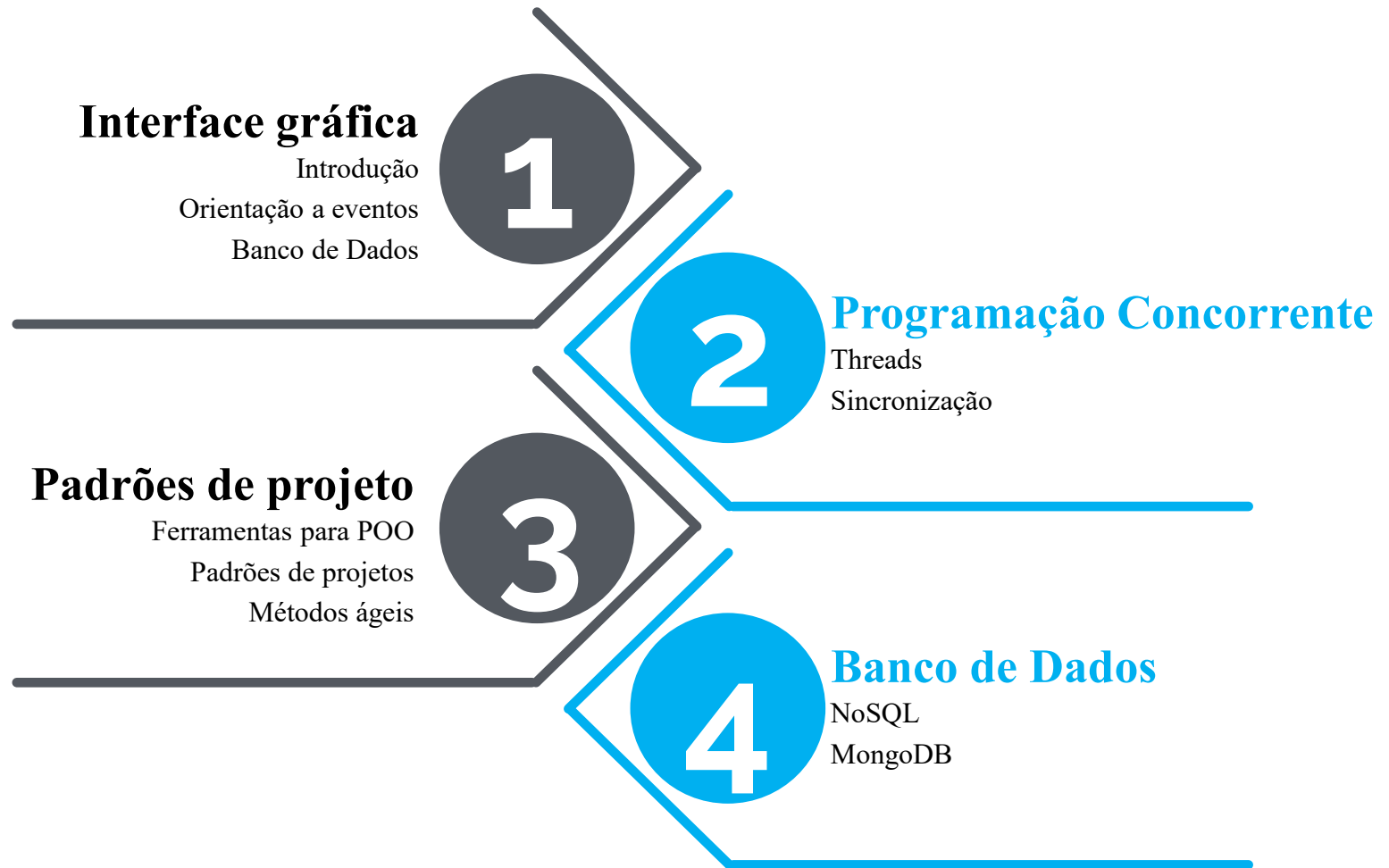


Programação Orientada a Objetos II

Profª. Angela Abreu Rosa de Sá, Dra.

Contato: angelaabreu@gmail.com

Ementa



Material Didático

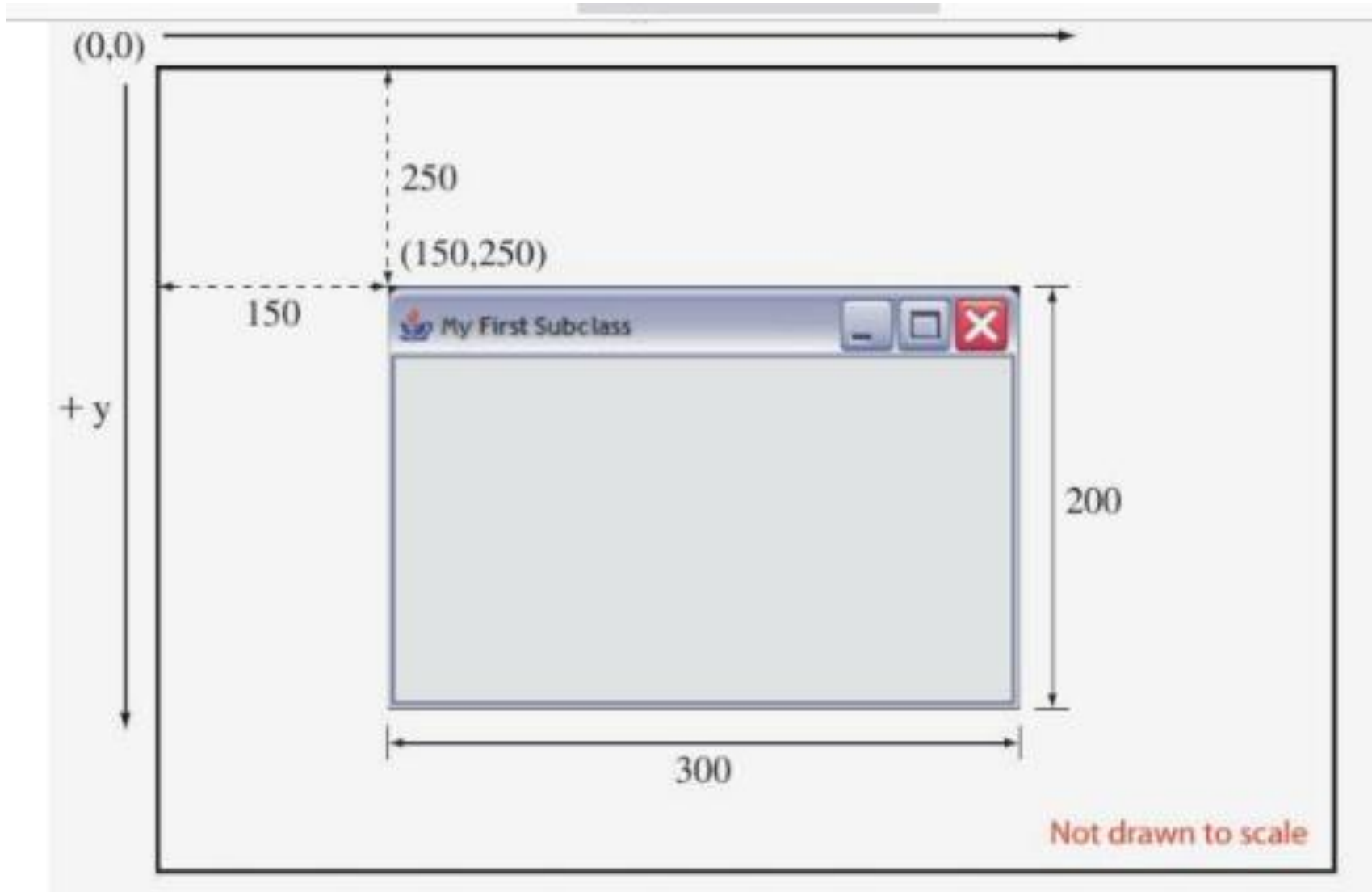


Programação Orientada a Objetos II

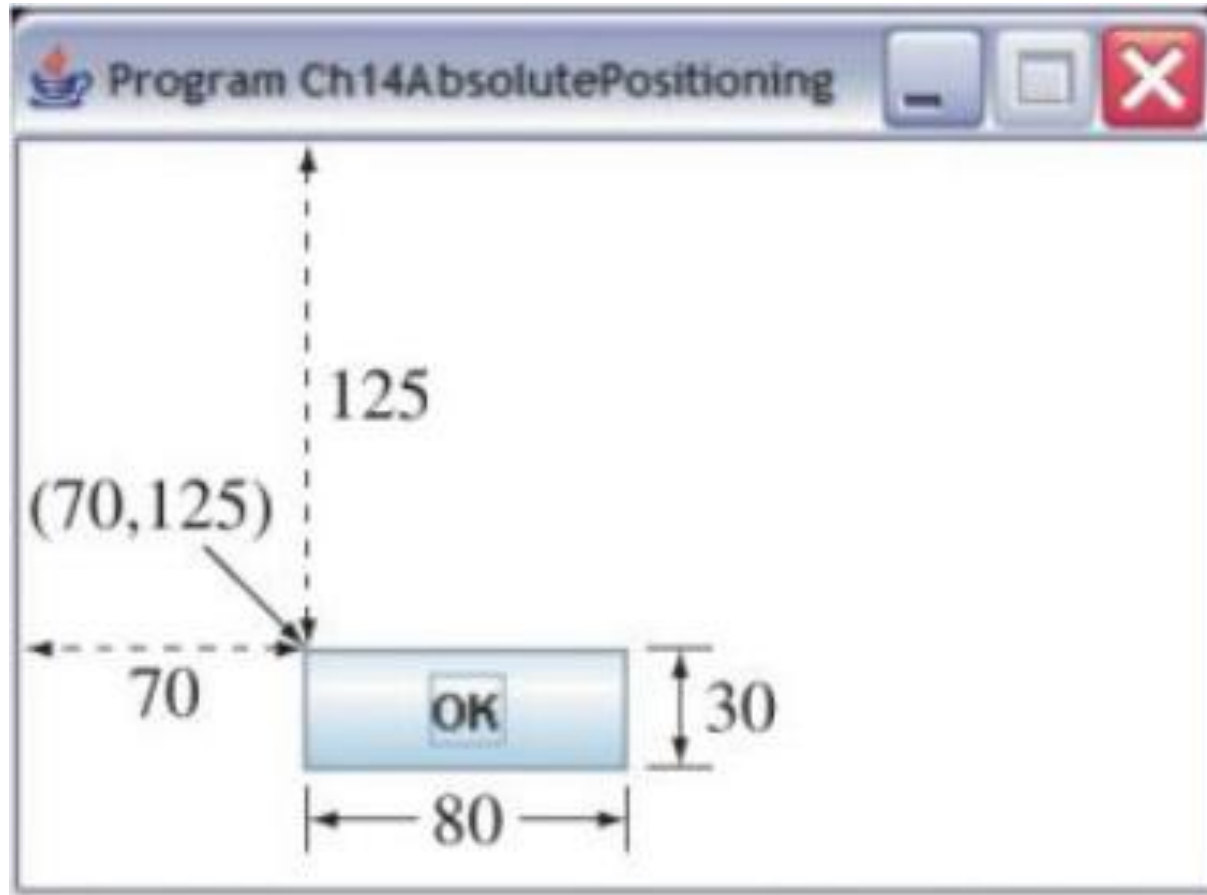
Sumário

Unidade 1 Programação orientada a eventos com interfaces gráficas e banco de dados relacional	7
Seção 1.1 - Desenvolvimento de interfaces gráficas na linguagem Java	9
Seção 1.2 - Programação em Java usando orientação a eventos	27
Seção 1.3 - Programação em Java usando banco de dados relacional	47
Unidade 2 Programação concorrente orientada a objetos	67
Seção 2.1 - Programação em Java usando threads	69
Seção 2.2 - Definição e tratamento de exceções para sistemas com threads	89
Seção 2.3 - Programação em Java utilizando elementos para sincronização em Java	107
Unidade 3 Padrões de projeto, ferramentas e métodos ágeis	127
Seção 3.1 - Ferramentas para programação em linguagens orientadas a objetos	129
Seção 3.2 - Padrões de projetos em orientação a objetos	149
Seção 3.3 - Métodos ágeis em orientação a objetos	167
Unidade 4 Novas tecnologias para programação em banco de dados	183
Seção 4.1 - Banco de dados NoSQL	185
Seção 4.2 - Introdução ao desenvolvimento em Java usando MongoDB	200
Seção 4.3 - Desenvolvimento em Java usando MongoDB	216

Interface Gráfica - SWING



Interface Gráfica - SWING



Interface Gráfica

The image shows a Java Swing window titled "Tela Inicial". Inside the window, there are three input fields and a button. The first field is labeled "Nome" and is a standard text field. The second field is labeled "CPF" and is a formatted text field with a mask. The third field is labeled "Tipo de usuário" and is a dropdown menu currently showing "Administrador". Below these fields is a large blue button labeled "Enviar".

→ **TextField**

→ **FormattedTextField**

→ **ComboBox**

→ **Button**

```
import java.awt.Container; //biblioteca para
containers

import java.text.ParseException;

import javax.swing.*; //simplificando a
inclusão de bibliotecas

import javax.swing.text.MaskFormatter;

public class PrimeiraTela extends JFrame {

    private JLabel lblNome;
    private JTextField txtNome;
    private JLabel lblCPF;
    private JFormattedTextField txtCPF;
    private JLabel lblTipo;
    private JComboBox cmbTipo;
    private final String[] tiposUsuarios =
    {"Administrador", "Geral"};
    private JButton btnOK;
    private Container ctn;

    public PrimeiraTela() {
        setSize(400, 300);
        setTitle("Tela Inicial");
        ctn = getContentPane();
        lblNome = new JLabel("Nome");
        txtNome = new JTextField();
        lblCPF = new JLabel("CPF");
        try {
            txtCPF = new JFormattedTextField(new
            MaskFormatter("###.###.###-##"));
        }
    }
}
```

```
    } catch (ParseException e) {  
        e.printStackTrace();  
    }  
    lblTipo = new JLabel("Tipo de usuário");  
    cmbTipo = new JComboBox(tiposUsuarios);  
    btnOK = new JButton("Enviar");  
    ctn.setLayout(null);  
    lblNome.setBounds(0, 0, 100, 25);  
    txtNome.setBounds(150,0,200,25);  
    lblCPF.setBounds(0,50,100,25);  
    txtCPF.setBounds(150,50,200,25);  
    lblTipo.setBounds(0,100,200,25);  
    cmbTipo.setBounds(150,100,200,25);  
    btnOK.setBounds(150, 150, 100, 100);  
    ctn.add(lblNome);  
    ctn.add(txtNome);  
    ctn.add(lblCPF);  
    ctn.add(txtCPF);  
    ctn.add(lblTipo);  
    ctn.add(cmbTipo);  
    ctn.add(btnOK);  
    setVisible(true);  
    setDefaultCloseOperation(JFrame.EXIT_  
ON_CLOSE);  
}
```

```
public static void main(String[] args) {  
    PrimeiraTela t1 = new PrimeiraTela();  
}
```

Tela Inicial

Nome

CPF

Tipo de usuário

Aulas - Aula01/src/view/PrimeiraTela2.java - Eclipse IDE

```
ctn = getContentPane();
lblNome = new JLabel("Nome");
txtNome = new JTextField();
lblCPF = new JLabel("CPF");
try {
    txtCPF = new JFormattedTextField(new MaskFormatter("###.###"))
}
catch (ParseException e) {
    e.printStackTrace();
}

lblTipo = new JLabel("Tipo de usuário");
cmbTipo = new JComboBox(tiposUsuarios);
btnOK = new JButton("Enviar");
ctn.setLayout(null);
lblNome.setBounds(0, 0, 100, 25);
txtNome.setBounds(150, 0, 200, 25);
lblCPF.setBounds(0, 50, 100, 25);
txtCPF.setBounds(150, 50, 200, 25);
lblTipo.setBounds(0, 100, 200, 25);
cmbTipo.setBounds(150, 100, 200, 25);
btnOK.setBounds(150, 150, 100, 100);
ctn.add(lblNome);
ctn.add(txtNome);
ctn.add(lblCPF);
ctn.add(txtCPF);
ctn.add(lblTipo);
ctn.add(cmbTipo);
ctn.add(btnOK);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public static void main(String[] args) {
    PrimeiraTela2 t1 = new PrimeiraTela2();
}

}
```

Problems @ Javadoc Declaration Coverage

Cuidado...

Os códigos vão ficando cada vez maiores!!!

- Organização;
- Entender cada linha do programa!
- Entender a sequencia lógica.

Material Didático

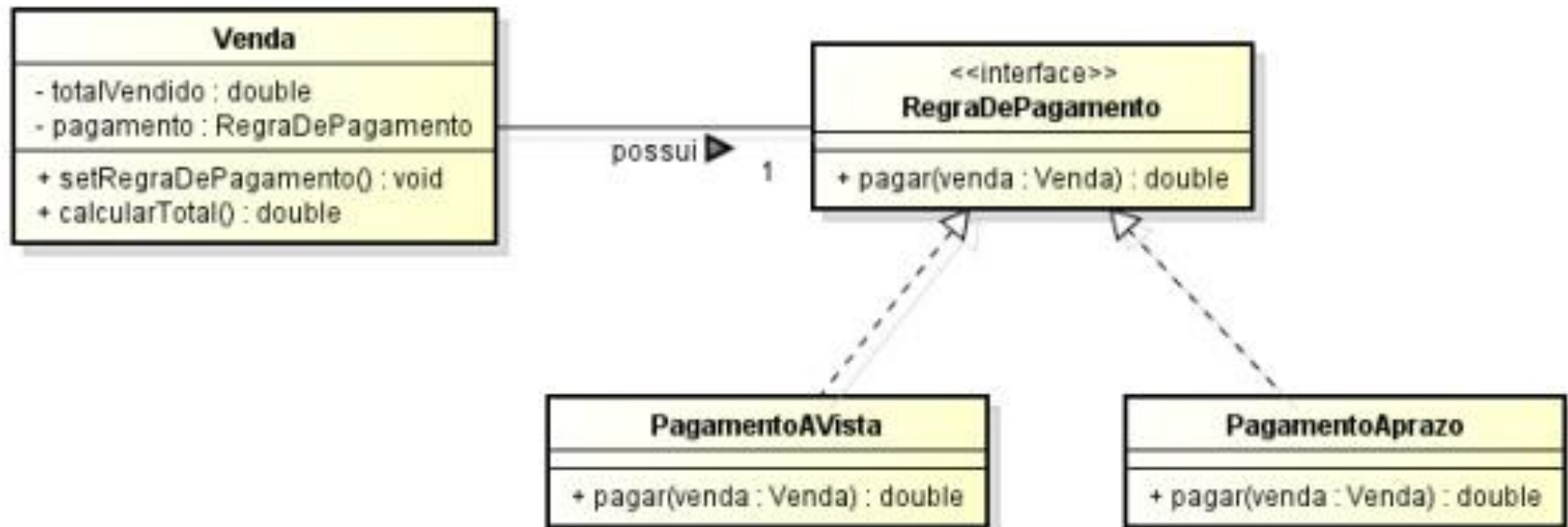


Programação Orientada a Objetos II

Sumário

Unidade 1 Programação orientada a eventos com interfaces gráficas e banco de dados relacional	7
Seção 1.1 - Desenvolvimento de interfaces gráficas na linguagem Java	9
Seção 1.2 - Programação em Java usando orientação a eventos	27
Seção 1.3 - Programação em Java usando banco de dados relacional	47
Unidade 2 Programação concorrente orientada a objetos	67
Seção 2.1 - Programação em Java usando threads	69
Seção 2.2 - Definição e tratamento de exceções para sistemas com threads	89
Seção 2.3 - Programação em Java utilizando elementos para sincronização em Java	107
Unidade 3 Padrões de projeto, ferramentas e métodos ágeis	127
Seção 3.1 - Ferramentas para programação em linguagens orientadas a objetos	129
Seção 3.2 - Padrões de projetos em orientação a objetos	149
Seção 3.3 - Métodos ágeis em orientação a objetos	167
Unidade 4 Novas tecnologias para programação em banco de dados	183
Seção 4.1 - Banco de dados NoSQL	185
Seção 4.2 - Introdução ao desenvolvimento em Java usando MongoDB	200
Seção 4.3 - Desenvolvimento em Java usando MongoDB	216

Interfaces



Interfaces

São coleções de métodos relacionados que informam aos objetos **o que estes devem fazer**, mas não como fazer.

As interfaces **são elementos que padronizam um conjunto de métodos**; dessa forma é possível que uma classe apresente **qualquer nível de complexidade**, e essa interface propicia as formas de fazer o **acesso padronizado e encapsulado**

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- Aula02
 - JRE System Library [JavaSE-17]
 - src
 - (default package)
 - AcessoElementos.java
 - Aluno.java

*AcessoElementos.java Aluno.java

```
1 import javax.swing.JFrame;
2
3 public class Aluno extends JFrame implements AcessoElementos {
4
5     @Override
6     public int getElemento(int index) {
7         // TODO Auto-generated method stub
8         return 0;
9     }
10
11     @Override
12     public void setElemento(int index) {
13         // TODO Auto-generated method stub
14     }
15 }
16
17 }
18
```

Problems @ Javadoc Declaration

0 errors, 2 warnings, 0 others

Description	Resource	Path	Location
Warnings (2 items)			

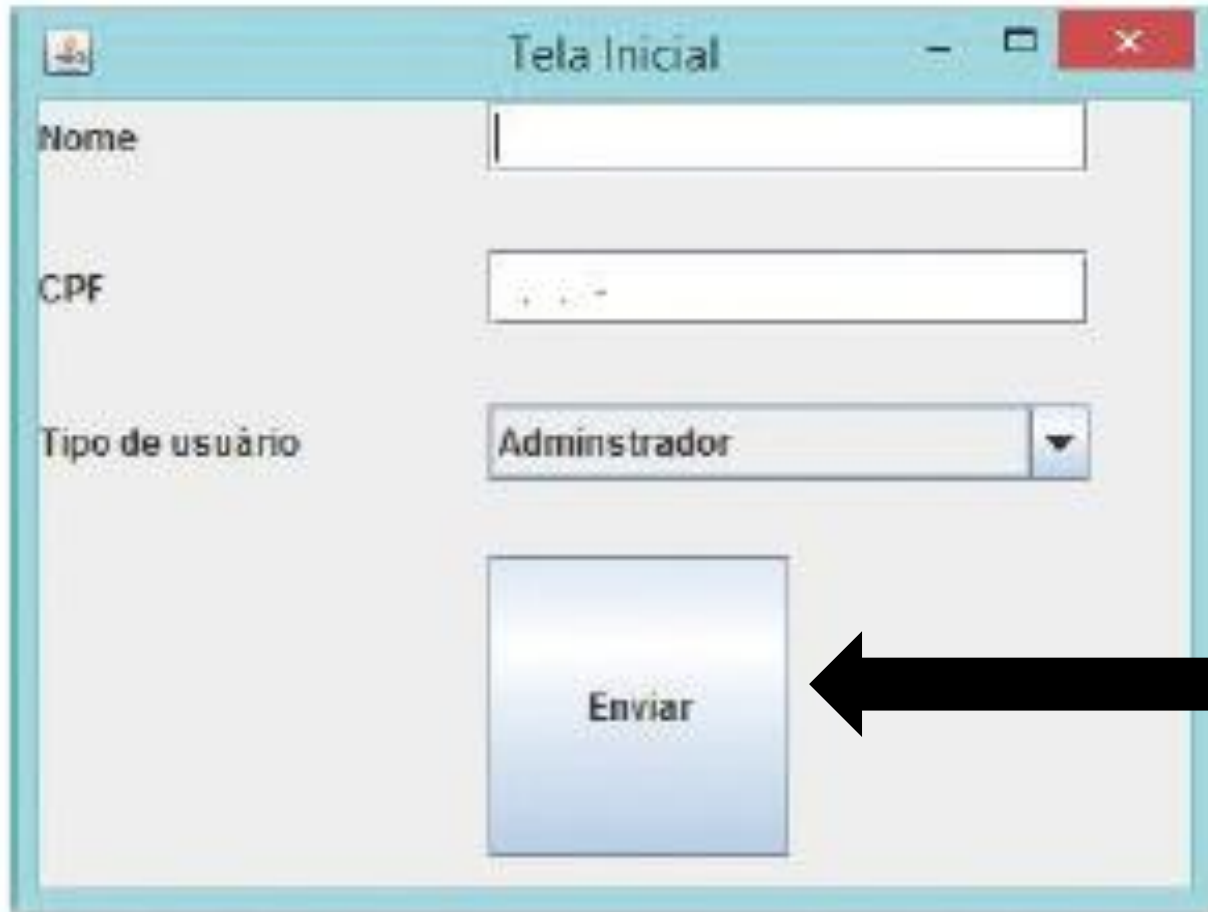
Override

Os métodos são sobrescritos

O que é um Evento?

acontecimento **gerado** observável

Interface Gráfica



The image shows a graphical user interface window titled "Tela Inicial". It contains three input fields: "Nome" (Name), "CPF" (CPF), and "Tipo de usuário" (User type). The "Tipo de usuário" field is a dropdown menu currently showing "Administrador". Below these fields is a large blue button labeled "Enviar" (Send). A black arrow points from the red text on the right towards the "Enviar" button.

Nome	
CPF	
Tipo de usuário	Administrador ▼
Enviar	

O que acontece ao clicar
no botão "Enviar"?

Orientação a Eventos

Na Interface Gráfica...

1. Criar uma relação de especialização com a classe que representa sua tela.
2. Declarar como atributos os elementos que serão adicionados à tela.
3. Definir a forma de alocação dos elementos gráficos na tela.
4. No construtor, instanciar, configurar e posicionar os itens na tela.
5. Tratar os eventos dos componentes para tratar as ações do usuário com a interface gráfica.

Tratamento de Eventos

Quadro 1.7 | Eventos na linguagem Java

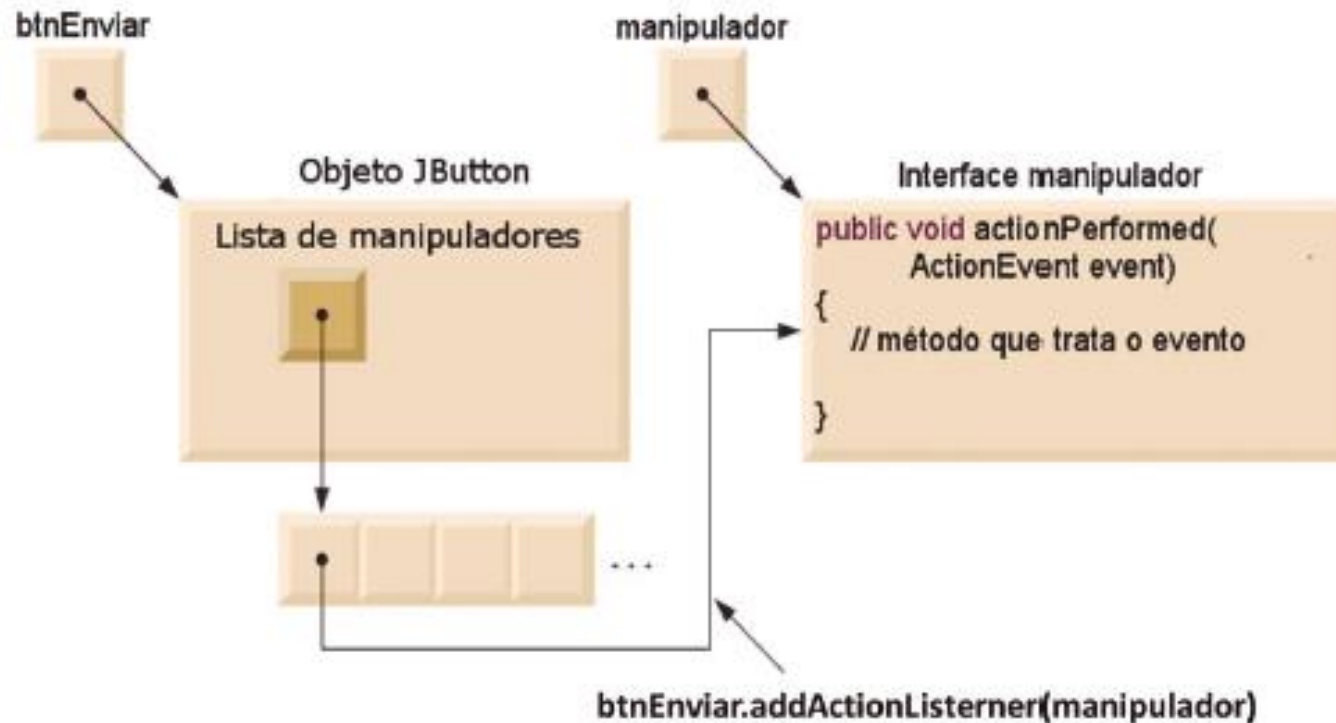


Evento	Descrição
ActionListener	Evento gerado pelo clique com o mouse em um botão.
ItemListener	Evento gerado quando um item em uma lista é selecionado.
FocusListener	Evento gerado quando um elemento ganha ou perde foco.
WindowListener	Evento gerado quando ocorre uma mudança na janela.

Fonte: elaborado pelo autor.

Interfaces

Figura 1.5 | Exemplo de como tratar o evento de clique em JButton do Java Swing



Fonte: adaptada de Deitel e Deitel (2016, p. 494).

Para implementar o tratamento de **eventos é necessário que a classe que está especializando o JFrame** tenha **alguma implementação dos métodos** da interface que trate os eventos e associe essa interface aos componentes que enviarão os eventos

Interfaces



O objetivo é verificar se o botão foi clicado pelo usuário, e para isso é possível utilizar a interface **ActionListener**.

Interfaces

Formas de implementar:



1

Implementar a interface `ActionListener` diretamente na classe que especializa o `JFrame`.

2

Fazer que cada componente crie uma classe anônima que implementa a interface `ActionListener`.

New Java Class

Java Class
Type already exists.

Source folder: Eventos/src Browse...

Package: (default) Browse...

☐ Enclosing type: Browse...

Name: PrimeiraTela

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: javax.swing.JFrame Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Finish Cancel

Implemented Interfaces Selection

Choose interfaces: actionli

Matching items:

ActionListener - java.awt.event - [jdk-18]

java.awt.event - [jdk-18]

Add OK Cancel

New Java Class

Java Class

✖ Type already exists.

Source folder: Eventos/src Browse...

Package: (default) Browse...

☐ Enclosing type: Browse...

Name: PrimeiraTela

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: javax.swing.JFrame Browse...

Interfaces: java.awt.event.ActionListener Add... Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Finish Cancel

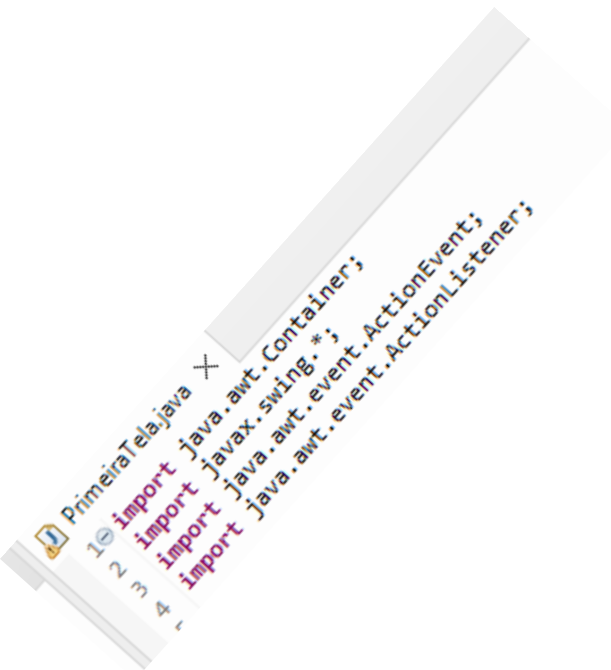
```
import java.awt.event.ActionEvent;

public class PrimeiraTela extends JFrame implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {
        // TODO Auto-generated method stub
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}
```

Interfaces



```
public class PrimeiraTela extends JFrame implements ActionListener
{
    private JButton btnok;
    private JTextField txtNome;
    private JLabel lblNome;
    private Container ctn;
    public PrimeiraTela()
    {
        setSize(300,140);
        setTitle("Eventos em Java Swing");
        ctn = getContentPane();
        ctn.setLayout(null);
        btnok = new JButton("Enviar");
        lblNome = new JLabel("Nome");
        txtNome = new JTextField();
        lblNome.setBounds(10,10,100,25);
        txtNome.setBounds(70,10,200,25);
        btnok.setBounds(90,50,80,40);
        ctn.add(lblNome);
        ctn.add(txtNome);
        ctn.add(btnok);
        // nesse ponto vamos inserir tratamento dos eventos
        btnok.addActionListener(this);

        setVisible(true);
    }

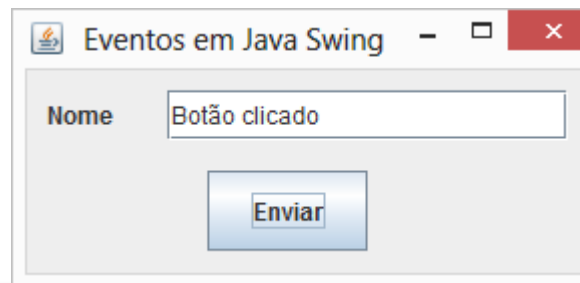
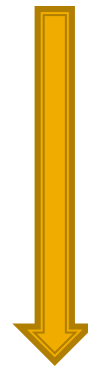
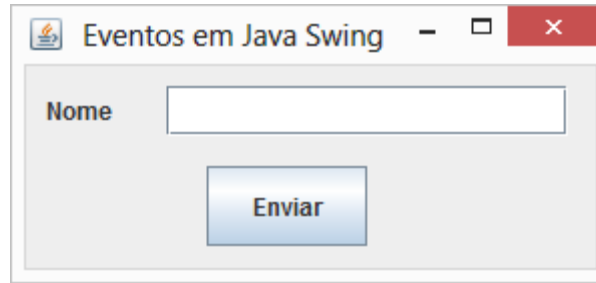
    public static void main(String[] args) {
        // TODO Auto-generated method stub

        PrimeiraTela tela = new PrimeiraTela();
    }

    //////////// implementação dos métodos dos eventos

    @Override
    public void actionPerformed(ActionEvent e)
    {
        if(e.getActionCommand().equals("Enviar"))
        {
            txtNome.setText("Botão clicado");
        }
    }
}
```

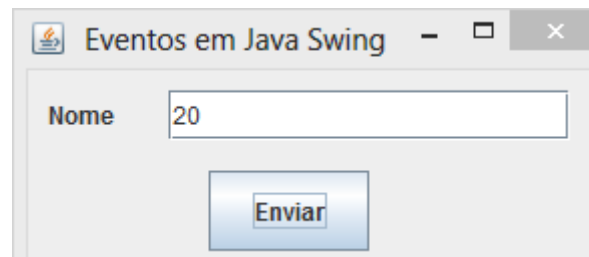
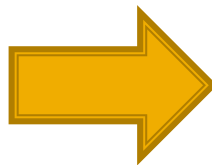
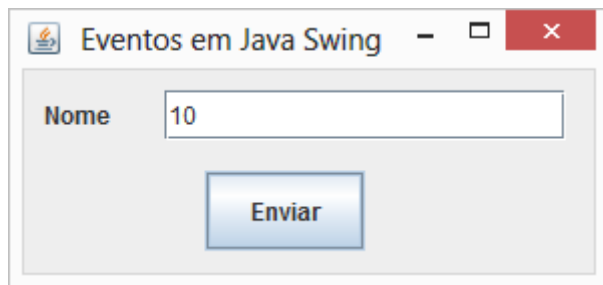
Interfaces



Interfaces

```
@Override
public void actionPerformed(ActionEvent e)
{
    if(e.getActionCommand().equals("Enviar"))
    {
        int num1 = Integer.parseInt(txtNome.getText());
        int num2 = num1+10;

        txtNome.setText(Integer.toString(num2));
    }
}
```



Interfaces

Formas de implementar:

1

Implementar a interface `ActionListener` diretamente na classe que especializa o `JFrame`.



2

Fazer que cada componente crie uma classe anônima que implementa a interface `ActionListener`.

New Java Class

Java Class

 The use of the default package is discouraged.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

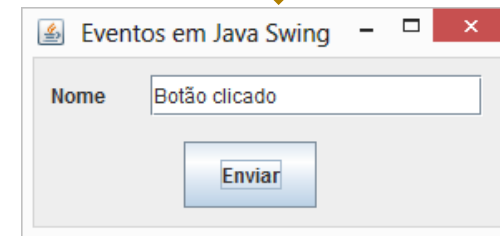
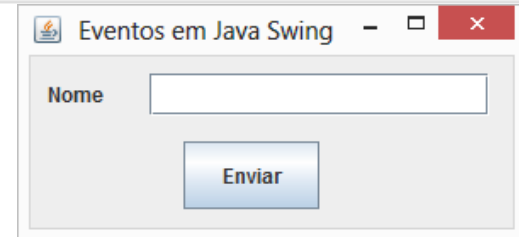
Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

```
1 import javax.swing.JFrame;
2
3 public class PrimeiraTela2 extends JFrame {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8     }
9
10 }
11
```

import java.awt.Container;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;



```
public class PrimeiraTela2 extends JFrame {  
  
    private JButton btnok;  
    private JTextField txtNome;  
    private JLabel lblNome;  
    private Container ctn;  
  
    public PrimeiraTela2()  
    {  
        setSize(300,140);  
        setTitle("Eventos em Java Swing");  
        ctn = getContentPane();  
        ctn.setLayout(null);  
        btnok = new JButton("Enviar");  
        lblNome = new JLabel("Nome");  
        txtNome = new JTextField();  
        lblNome.setBounds(10,10,100,25);  
        txtNome.setBounds(70,10,200,25);  
        btnok.setBounds(90,50,80,40);  
        ctn.add(lblNome);  
        ctn.add(txtNome);  
        ctn.add(btnok);  
        // nesse ponto vamos inserir tratamento dos eventos criando uma classe anônima  
        btnok.addActionListener(new ActionListener()  
        {  
            public void actionPerformed(ActionEvent e)  
            {  
                trataBotaoOk();  
            }  
        });  
  
        setVisible(true);  
    }  
  
    public void trataBotaoOk()  
    {  
        txtNome.setText("Botão clicado");  
    }  
  
    public static void main(String[] args) {  
        PrimeiraTela2 tela = new PrimeiraTela2();  
    }  
}
```

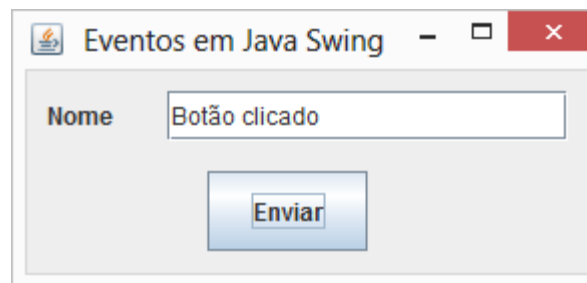
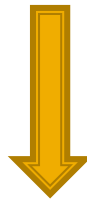
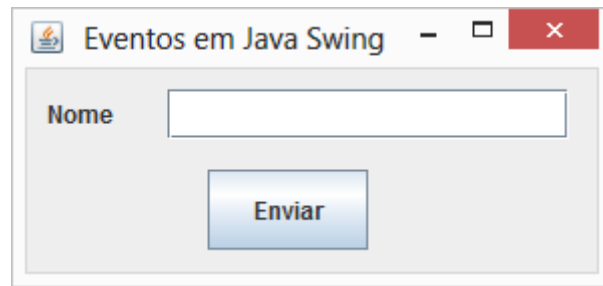




Exercícios 😊

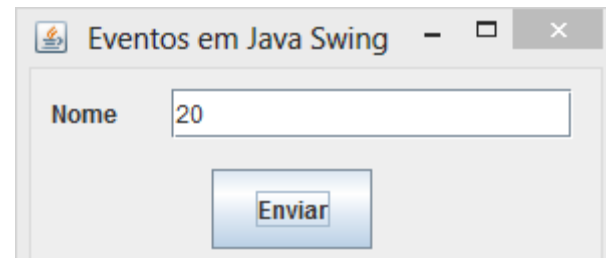
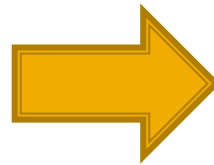
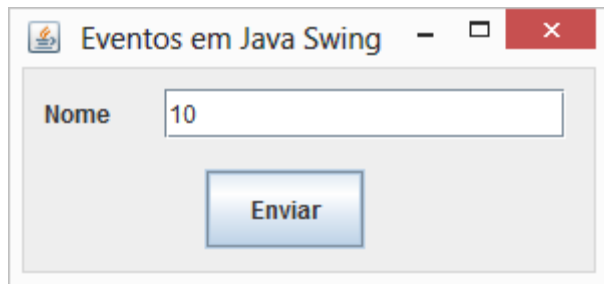
Exercício 1

Implementar o evento abaixo utilizando **o método 1**: implementar a interface `ActionListener` diretamente na classe que especializa o `JFrame`

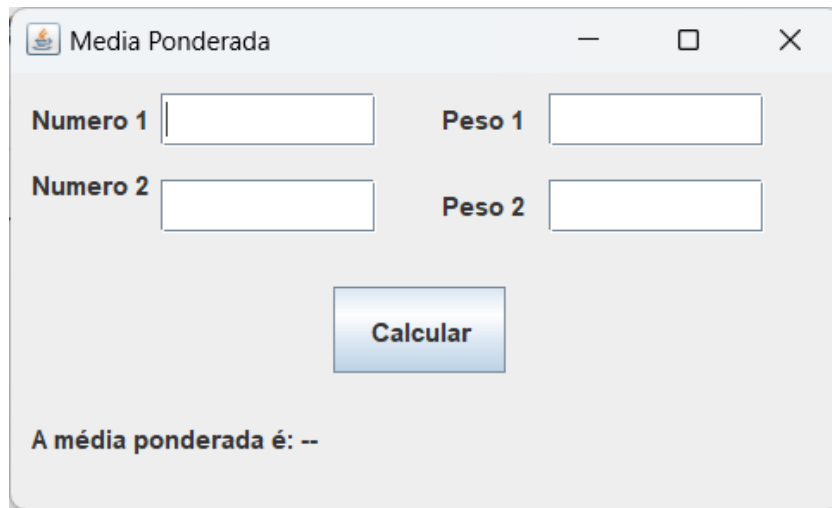


Exercício 2

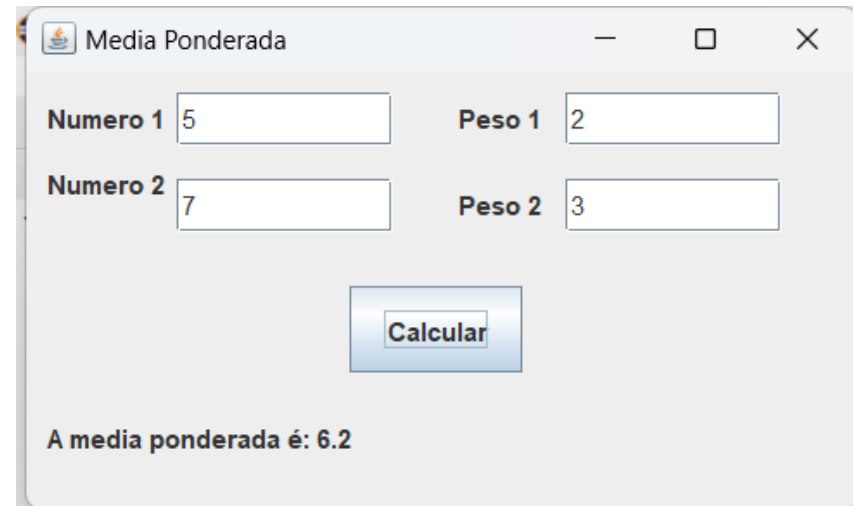
Implementar o evento abaixo utilizando **o método 2**: criar classe anônima que implementa a interface ActionListener



Exercício 3



The screenshot shows a window titled "Media Ponderada" with standard Windows window controls (minimize, maximize, close). It contains two input fields for "Numero 1" and "Numero 2", and two input fields for "Peso 1" and "Peso 2". A blue "Calcular" button is centered below the input fields. At the bottom, the text "A média ponderada é: --" is displayed.



The screenshot shows the same "Media Ponderada" window after calculation. The input fields now contain the values: "Numero 1" is 5, "Numero 2" is 7, "Peso 1" is 2, and "Peso 2" is 3. The "Calcular" button remains. The text at the bottom now reads "A media ponderada é: 6.2".

$$\text{Média Ponderada} = \frac{(\text{Numero1} * \text{Peso1}) + (\text{Numero2} * \text{Peso2})}{(\text{Peso 1} + \text{Peso2})}$$

```
@Override
public void actionPerformed(ActionEvent e) {

    if(e.getActionCommand().equals("Calcular"))
    {

        float num1 = Float.parseFloat(txtNum1.getText());

        float num2 = Float.parseFloat(txtNum2.getText());

        float peso1 = Float.parseFloat(txtPeso1.getText());

        float peso2 = Float.parseFloat(txtPeso2.getText());

        float media = ((num1 * peso1) + (num2 * peso2)) / (peso1 + peso2);

        lblMediaPonderada.setText("A media ponderada é: " + Float.toString(media));

    }

}
```

Exercício 4

Implemente uma interface gráfica que receba o **peso** e a **altura** e uma pessoa e calcule o Índice de Massa Corporal (IMC) :

$$IMC = peso / (altura * altura).$$

Ao clicar no botão “Calcular”, o programa deverá exibir na tela:

- O Valor do IMC
- Mensagem:
 - “Peso Ideal”, caso o IMC esteja entre 18.5 e 25;
 - “Você está acima do peso ideal!”, caso o IMC esteja acima de 25;
 - “Você está abaixo do peso ideal”, caso o IMC esteja abaixo de 18.5.

Implemente a **Classe IMC**. A Interface Gráfica deverá instanciar um objetivo da classe IMC para realizar as operações necessárias.



Muito Obrigada!

Profª. Angela Abreu Rosa de Sá, Drª.

Contato: angelaabreu@gmail.com